# A Scale-Aware, GPU-Resident Framework for Real-Time Small-Scale Liquid Simulation and Rendering

Research design, numerical methods, and reproducible evaluation protocol

**FIGURE PLACEHOLDER**

`figures/teaser.png`

Replace with: microdroplet on inclined glass; coalescence sequence; continuous thin filament; matching final photorealistic render. No results are implied.

**Figure 1: Target phenomena and visual fidelity (illustration pending).** Four proposed evaluations at sub-millimetre to centimetre scales. The panel is intentionally empty until implementations produce measured, reproducible frames. No image in this manuscript is presented as an achieved result.

**Abstract** Achieving physically plausible, visually continuous liquids on constrained GPUs is especially difficult at small spatial scales, where capillarity, contact-line hysteresis, surface adhesion, and boundary friction can dominate gravity. Particle solvers that appear adequate for large splashes can produce unstable, ballistic microdroplets; conversely, high-fidelity implicit solvers can require computation far beyond a real-time budget. This paper specifies a scale-aware, GPU-resident framework that decouples physical resolution from optical reconstruction, compares position-based and divergence-free pressure projections against a unified pressure–surface-tension–friction formulation, and prioritizes droplet-scale validation before scene-scale optimization. The proposed system uses compact GPU neighbor indexing, persistent simulation buffers, model-based free-surface reconstruction, and a thickness-aware optical compositor. We identify the gap between published offline examples and the desired 60 Hz frame time, define measurable numerical and rendering acceptance tests, and provide an implementation sequence designed for a minimal Vulkan abstraction with an Apple M2 baseline. The present contribution is a *research-backed design and evaluation protocol*; it does not claim a completed fluid solver, experimental accuracy, or measured real-time performance.

**CCS Concepts:** Computing methodologies → Physical simulation; Rendering; Massively parallel algorithms.
**Keywords:** smoothed particle hydrodynamics, droplets, capillarity, surface tension, contact-line hysteresis, GPU compute, small-scale fluids, physically based rendering.

## 1 Introduction

Real-time fluid effects frequently optimize for impressive large-scale motion while under-resolving the liquid interfaces that control tiny droplets, wetting, capillary bridges, and thin streams. At these scales, liquid appearance is a coupled numerical and optical problem. A particle cluster can preserve approximately the correct mass yet look like beads; a smooth reconstructed surface can be visually convincing while violating contact-angle or pressure constraints. Neither compromise is acceptable as the sole success criterion of a physically motivated system.

The design target is a reusable liquid library, not a blood-specific effect. It should support water-like Newtonian liquids first, then measured non-Newtonian materials if the solver and benchmarks warrant the added complexity. Inputs are SI material properties, conservative emission volumes, collider geometry, boundary parameters, and optical coefficients. The simulation must remain independent of game-specific scene generation, collision semantics and control systems.

The requirements are stringent but not contradictory when explicitly bounded: visually continuous isolated droplets and filaments, observable conservation, stable interaction with solid surfaces, and a 60 Hz display target on a fanless Apple M2-class GPU. What is *not* feasible is a universal guarantee of 60 Hz for arbitrary numbers of sub-millimetre particles, resolutions, or contact constraints.

We make four design propositions. First, small-scale liquid physics needs measured pressure and capillary behavior, not hand-tuned ejection velocities. Second, all large particle and neighbor arrays should remain GPU-resident. Third, the representation used to shade a droplet may be finer than the simulation sampling without inventing physical mass. Fourth, algorithms should be selected by quantified error and measured cost rather than by the visual quality of a single demonstration.

## 2 Related Work

### 2.1 Incompressible particle solvers

Implicit Incompressible SPH (IISPH) derives a pressure projection from the continuity equation and pressure force to improve convergence [1]. Divergence-Free SPH (DFSPH) further applies separate density and velocity-divergence corrections, reducing compressibility and permitting larger timesteps in tested cases [2]. Position Based Fluids (PBF) formulates density correction as a positional constraint and is attractive for parallel real-time systems [3]. GPU PCISPH implementations demonstrate that indexing and memory layout can be as important as the pressure model [4, 5].

None of these methods alone guarantees realistic small-

droplet contact and adhesion: their pressure constraints do not uniquely determine hysteresis, friction, free-surface shape, or the behavior of a thin liquid bridge. Each is therefore a reference to compare, not a complete target implementation.

## 2.2 Tension, adhesion, and friction

Akinci et al. provide a widely used surface-tension and adhesion SPH treatment [6]. Probst and Teschner derive a unified implicit pressure, surface-tension, and Coulomb-like boundary-friction solver [7], building on work on monolithic friction and contact handling [8]. The unified method is a compelling physical target for droplets on tilted or vertical solids.

**Critical runtime distinction.** The 2024 paper reports large-scale demonstrations whose computation time is measured in seconds *per simulated step*, not 60 interactive steps per second [7]. Transferring the underlying physics to compact GPU kernels is an unverified engineering research task. We neither claim to reproduce its exact convergence nor use its offline throughput to predict M2 frame rates.

## 2.3 Small free surfaces and optical rendering

Yu and Turk's anisotropic reconstruction uses local particle covariance to preserve thin streams and sharp features [9]. Ando et al. apply adaptive anisotropic resampling to maintain thin sheets in a FLIP context [10]. Screen-space curvature-flow smoothing can reconstruct fluid surfaces efficiently while suppressing sphere-like artifacts [11]. These studies motivate a hybrid reconstruction that preserves isolated droplets rather than smoothing every particle group equally.

Physical thin films over curved surfaces can alternatively use reduced surface PDEs with conservative transport [12]. We do not automatically add a 2-D film solver: its coupling, geometry updates, and small contact physics must be justified against the cost and quality of the primary 3-D approach. APIC is retained as an optional bulk-liquid extension, not the starting point for capillary-dominated droplets [13].

# 3 Scale Regimes and Dimensional Analysis

## 3.1 Dimensionless controls

Let $\rho$ be density (kg/m$^3$), $\mu$ dynamic viscosity (Pa s), $\sigma$ surface tension (N/m), $U$ a characteristic speed, $L$ a characteristic length, and $g$ gravity. We use

$$\mathrm{Re} = \frac{\rho U L}{\mu}, \qquad \mathrm{We} = \frac{\rho U^2 L}{\sigma}, \tag{1}$$

$$\mathrm{Bo} = \frac{\rho g L^2}{\sigma}, \quad \mathrm{Oh} = \frac{\mu}{\sqrt{\rho \sigma L}}, \quad \mathrm{Ca} = \frac{\mu U}{\sigma}. \tag{2}$$

The capillary length

$$\ell_c = \sqrt{\frac{\sigma}{\rho g}} \tag{3}$$

is approximately 2.71 mm for water near room temperature using $\rho = 998\,\mathrm{kg/m^3}$ and $\sigma = 0.072\,\mathrm{N/m}$. For droplets markedly below that length scale, capillarity is typically important relative to gravity. This is a scaling criterion, not a fixed particle-spacing prescription.

**FIGURE PLACEHOLDER**

`figures/scale-regimes.png`

Add a log-scale plot of Bo and Oh versus droplet radius, marking the intended resolution bands and the measured numerical stability limits.

**Figure 2: Scale regimes and solver resolution (planned).** Analytic dimensionless curves are distinct from measured stability and error. Figure must be populated from reproducible scripts.

For a static spherical interface, the Young–Laplace pressure jump is

$$\Delta p = \frac{2\sigma}{R}. \tag{4}$$

At $R = 0.25$ mm the prediction for the nominal water properties is 576 Pa; at $R = 1$ mm, 144 Pa. These analytic values are *reference targets*, not simulated measurements.

## 3.2 Spatial resolution cannot be hidden

A particle simulation only resolves interfaces down to a scale tied to its kernel support and particle spacing $\Delta x$. If a filament becomes thinner than the neighborhood support, smoothing can dissolve it and arbitrary attraction can make it too thick. The renderer can display a plausible sub-pixel droplet at physical volume $V$ using an analytic shape, but this is *not* evidence that internal capillary waves, rupture, or contact-angle dynamics at that scale were numerically resolved.

We therefore report two independent quality measures: *simulation fidelity*, determined by $\Delta x$, solver residual, and interface convergence, and *display fidelity*, determined by projected area, visibility, normals, thickness, and optical sampling.

## 3.3 Time-step constraints

A physical substep is controlled by advection, pressure response and capillary dynamics. Useful *diagnostic* scales are

$$\Delta t_{\mathrm{adv}} \sim \frac{h}{\|\mathbf{v}\| + \epsilon}, \qquad \Delta t_\sigma \sim \sqrt{\frac{\rho h^3}{\sigma}}, \tag{5}$$

where $h$ is characteristic kernel support. Constants and stability limits depend on discretization and implicitness; Eq. (5) is not an asserted stable timestep for the proposed solver. A stiff implicit capillary solve may tolerate larger steps, but must prove convergence rather than silently dropping steps to meet frame rate.

# 4 Proposed GPU Architecture

## 4.1 System boundary

The implementation should remain a static C++17 library under `fluid/fld/`, with Vulkan operations expressed through the existing VulkanEngine abstraction. Public code supplies material properties, emission requests, collider state,

camera matrices, and command recording. No CUDA dependency, engine-embedded game controls, or user-facing blood-specific API is permitted.

At steady state, the CPU should allocate no per-particle arrays, traverse no particle neighborhoods, and read back no positions to construct rendering bins. GPU buffers are persistent, capacity-aware, and bound per frame. Host uploads consist of small emission, material, and camera parameter queues.

## 4.2 Compact particle representation

The core state is a structure-of-arrays (SoA): positions, velocities, masses, phases, local pressure, boundary/contact state, and sorted IDs. One plausible minimal layout stores two `vec4` buffers for position/radius and velocity/phase, with scalar state in aligned arrays. Encoding a phase as floating-point data without integer round-trip guarantees is avoided. An initial implementation can use 32-bit floats with a local scene origin; large-coordinate worlds require rebasing.

The pipeline is

$$\boxed{\begin{array}{c}\text{Emit/Compact} \to \text{Build Cell Keys} \to \text{Count/Scan/Scatter} \\ \to \text{Neighbors} \to \text{Implicit Fluid Step} \to \text{Contact} \\ \to \text{Surface Classification} \to \text{Optical Passes.}\end{array}} \tag{6}$$

## 4.3 Neighbor indexing: dense versus sparse domains

For bounded uniform-resolution neighborhoods, choose cell width $c \geq h$ and inspect the current plus up to 26 adjacent cells. A dense index uses one count per active-domain grid cell, an exclusive prefix scan over counts, and one scatter pass to sort particle IDs into contiguous ranges. This permits contiguous iteration and avoids CPU-managed lists [4, 14].

A naive dense grid is unsuitable for extremely sparse, spatially dispersed microdroplets. The indexer must therefore expose either (a) a tightly bounded active region with measured occupancy or (b) a sorted sparse key stream and cell-range table, with a direct comparison of both options. Hash collisions must be resolved by comparing *full cell coordinates*, never silently combining unrelated cells.

Let $N$ be live particles and $K$ candidate cell entries. In a practical implementation, indexing is approximately $O(N + K)$ for a bounded histogram, while neighbor evaluation remains $O(N\bar{n})$ for mean neighborhood size $\bar{n}$. Claims of globally $O(N)$ fluid simulation are misleading if $\bar{n}$ grows with compression, multiscale support, or dense surface packing.

A work-efficient scan, e.g., a hierarchical Blelloch-style scan or decoupled look-back subject to cross-workgroup progress assumptions, is suitable for the prefix stage [15]. We begin with a portable multi-dispatch scan; more aggressive device-wide scans are adopted only after validating memory-order semantics and measuring benefit on MoltenVK.

## 4.4 Deterministic GPU stages

An optional validation mode uses stable keys and a deterministic sort order to reproduce neighbor sets across runs. Floating-point reductions can still vary across devices, so bitwise identity between unrelated GPUs is not an acceptance criterion. For release mode, atomics and scatter order may differ, but the simulator must bound the change in physical outputs.

Each active buffer records count, capacity, overflow status, and generation. Overflow may reject a new emission *with its physical mass still pending*, never silently truncate existing liquid. The emitter can implement backpressure or report a refused volume.

# 5 Numerical Solver

## 5.1 Reference incompressibility

SPH estimates density at particle $i$ by

$$\rho_i = \sum_j m_j W(\mathbf{x}_i - \mathbf{x}_j, h), \tag{7}$$

with a normalized compact-support kernel $W$. The pressure constraint targets $\rho_i/\rho_0 - 1 \approx 0$ away from free-surface deficits. Density and velocity-divergence residuals are separate acceptance metrics. We first implement an inexpensive GPU PBF baseline, followed by DFSPH/IISPH comparisons [1–3].

The baseline is a *measurement instrument*: identical particles, scenes, neighbor index, contact geometry, time budget, and residual tolerances are used for all candidates. Iterations are not compared without reporting their actual convergence and memory traffic.

## 5.2 Coupled capillarity and friction

The desired physical extension follows the motivation of Probst and Teschner [7]. The normal pressure response, cohesive interface forces, and solid-boundary friction should be solved in a coupled way whenever the interface is capillary-dominated. A simple explicit attraction between particles is *not* treated as a validated substitute for this method.

At a solid, static wetting requires both advancing and receding contact angles. The pinning resistance scales with a contact-line factor of the form

$$F_{\text{pin}} \sim \sigma w \left(\cos\theta_R - \cos\theta_A\right), \tag{8}$$

where $w$ is an effective contact-line width. This is a Furmidge-type macroscopic scaling law [16], not a direct prescription for the per-particle friction impulse. Mapping the law to a consistent discretization must be calibrated on resting and sliding droplets.

For each collider, the interface supplies signed distance or closest-point queries, normal, relative surface velocity, material identity, advancing/receding angles, and friction parameters. Two-way momentum exchange with moving rigid bodies is optional but, when enabled, must conserve equal and opposite impulses within numerical tolerance.

## 5.3 Convergence and stability gates

For a candidate step, require finite state, bounded density residual, bounded divergence, nonnegative mass, reasonable energy growth, and low penetration. If any condition fails, retry with smaller $\Delta t$ or report an invalid simulation step; do not render corrupted state as a valid result. We explicitly retain

**FIGURE PLACEHOLDER**

`figures/pipeline.png`

Insert implementation block diagram: GPU buffers, emission, compact hash, scan, pressure/tension/friction iterations, render passes, with explicit synchronization edges.

**Figure 3: GPU-resident simulation and rendering pipeline (architecture placeholder).** Colored regions should eventually indicate measured stage costs and buffer ownership. No implementation timings are currently available.

**FIGURE PLACEHOLDER**

`figures/laplace-validation.png`

Insert measured mean pressure jump versus inverse droplet radius for at least three spatial resolutions; overlay Young–Laplace reference.

**Figure 4: Laplace-pressure validation (results pending).** The diagonal reference must use Eq. (4); measured points are to be inserted only after pressure sampling and convergence testing.

a validation reference on the CPU for a *small* deterministic particle configuration, but not as a per-frame production solver.

A droplet that stands still on an incline is not necessarily correct: artificial velocity damping can mimic pinning. Compare sliding onset across droplet sizes and surface angles with a contact-angle hysteresis model. A spherical droplet passing Eq. (4) but failing merging is likewise insufficient.

## 6 Microscopic Droplets and Filaments

### 6.1 Isolated droplet mode

Rendering an isolated droplet as a pixel-sized sphere of fixed radius is visually misleading. Instead, infer its displayed diameter from conserved volume and a shape family constrained by contact angle and substrate orientation. For a freely suspended spherical drop, $R = (3V/4\pi)^{1/3}$. For a sessile drop, use a spherical-cap solution only where gravity is negligible; for larger Bo, numerical Young–Laplace profiles are a better reference [16].

This is a *render-side reconstruction*, not an exemption from physics. If the particle solver has insufficient samples to model coalescence or breakup, those interactions must be labeled approximate. The renderer must never transform an arbitrary particle cluster into an unsupported physical claim.

### 6.2 Thin-thread survival

A stream one or two particle layers across can disappear under isotropic density smoothing. Adaptive anisotropic kernels [9] offer a means to preserve narrow geometry. For each surface candidate, compute a local covariance

$$\mathbf{C}_i = \sum_j w_{ij}(\mathbf{x}_j - \bar{\mathbf{x}}_i)(\mathbf{x}_j - \bar{\mathbf{x}}_i)^T, \tag{9}$$

whose regularized eigensystem defines an ellipsoidal reconstruction kernel. Extreme aspect ratios are clamped to avoid exploding surface normals or connecting physically disjoint drops.

Adaptive particle splitting, as in the FLIP work of Ando et al. [10], is an option only when each split preserves parent mass and momentum and the increased particle count is tracked. It must not be activated merely because a thin stream is visible on screen.

### 6.3 Coalescence versus separation

Two nearby droplets should merge when their free surfaces establish a genuine liquid bridge under the chosen physical model. Conversely, distinct small drops must remain distinct when separated by a non-negligible air gap. The renderer's connectivity classifier must therefore use particle-neighbor geometry, narrow-gap heuristics and phase identity; screen-space overlap alone is insufficient.

## 7 Photorealistic Surface Reconstruction

### 7.1 Reconstruction classes

The renderer treats isolated microdroplets and connected free liquid differently. The first class uses an analytic or tightly fitted meniscus contour with stable normals and an explicit volume-to-shape relationship. The second class uses covariance-aware particle kernels or a screen-space surface representation appropriate to the projected footprint [9, 11].

We require explicit guards for (i) single-particle drops, (ii) filament-like neighborhoods, (iii) merging sheets, (iv) disocclusion, and (v) very shallow fluid against scene geometry. An isotropic gaussian field cannot simultaneously preserve all five cases without tuned, view-dependent artifacts.

### 7.2 Optics

We distinguish front-surface depth, back-surface depth (or integrated thickness), scene depth and scene color. Transmittance is approximated with the Beer–Lambert law,

$$\mathbf{T} = \exp(-\boldsymbol{\sigma}_a d), \tag{10}$$

**FIGURE PLACEHOLDER**

`figures/microdrop-sequence.png`

Insert paired quantitative+visual sequences: (a) isolated 0.5 mm droplet, (b) two-drop coalescence, (c) contact-line pinning, (d) 1 mm filament. Include time labels and scale bars.

**Figure 5: Small-scale physics comparison (images pending).** Each panel should show a simulation frame and a reference photograph or validated offline solution using the same physical scale and material parameters.

**FIGURE PLACEHOLDER**

`figures/rendering-ablation.png`

Insert 2 x 2 comparison: raw spheres, isotropic reconstruction, anisotropic reconstruction, final Beer–Lambert + Fresnel render, matched camera and volume.

**Figure 6: Rendering ablation (images pending).** The figure must isolate the benefit of each reconstruction and optical stage; no unmeasured quality rankings are asserted.

where $d$ is liquid path length and $\boldsymbol{\sigma}_a$ is spectral or three-channel absorption. Refraction uses the liquid's index of refraction and available scene color/depth; screen-space refraction cannot see obscured off-screen geometry and must have a documented fallback. Reflection uses Fresnel terms, consistent roughness and environment lighting. This is an *approximate real-time optical model*, not full path tracing.

Normal smoothing is edge-aware. Do not blur across a droplet/background boundary or fuse independent drops solely because their depth buffers overlap. We initially target temporal stability without mandatory TAA, since accumulated history can smear small fast features.

## 8 Real-Time GPU Engineering

### 8.1 Memory and bandwidth

The main stage costs are sorted-neighbor traffic, iterative field evaluation, boundary queries, and optical images. Assume, provisionally, that a particle consumes 64–128 bytes of persistent working state across buffers. At 100,000 particles, this corresponds to approximately 6.4–12.8 MB, *excluding* ping-pong arrays, sort buffers, image targets, material tables, allocator metadata and geometry acceleration. Actual footprint will be recorded from allocations, not estimated from structure sizes alone.

Particles are emitted and removed via GPU compaction and free-list accounting. Fixed-size allocations avoid per-frame churn; the graphics system may grow capacity between frames after ensuring frame safety. If buffers are shared by compute and render, synchronization must explicitly enforce writes-before-reads with the correct stages and access masks [17].

### 8.2 M2 and MoltenVK considerations

Apple GPUs are tile-based renderers; Apple documents local tile-processing and traffic reduction principles [18, 19]. A Vulkan renderer running through MoltenVK may not expose every Metal-specific optimization, so no Metal imageblock feature is assumed usable from the generic engine API. Workgroup size, subgroup support, buffer memory model and storage-image formats are queried from the actual device, not hardcoded from desktop GPUs.

The initial GPU indexer should have a portable baseline using 64 or 128 threads per workgroup and independent prefix-scan dispatches. The best configuration is selected from measurements, with barriers between histogram, scan, scatter, pressure iterations, and rendering. Avoid full-device waits and unnecessary CPU readbacks. Queries and frame-safe delayed readback report GPU timings without introducing permanent stalls.

### 8.3 Quality preserving adaptation

If the physics solver cannot complete the required substeps, the system should *not* silently lower pressure iterations, discard liquid or coarsen particles. Instead, the host receives simulation backlog and quality status. Separately adjustable rendering-resolution and anisotropy quality may be reduced if screen-space error metrics remain below thresholds; these adaptations must not change physical mass or accepted pressure residuals.

The 60 Hz target is defined for an explicit bounded scene, viewport, liquid volume and hardware temperature range. The measured p99 frame time must be below 16.67 ms, with sustained simulation-time progression and no dropped physical steps. For some micro-scale scenes this may be impossible with the chosen exact solver; that negative finding must be reported rather than hidden.

## 9 Evaluation Protocol

### 9.1 Reproducibility

Every run records commit hash, compiler flags, GPU device/driver, MoltenVK and Vulkan versions, shader hashes, resolution, render scale, material constants, particle spacing/support, timestep policy, input seed, warmed-up interval,

**Table 1: Required tests and status.** All performance cells are intentionally unfilled until measurements exist.

| Test | Required observation / measurement |
|---|---|
| Drop, $R = 0.25$–$1$ mm | $\Delta p(R)$, radius drift, density residual |
| Inclined drop | $\theta_A$, $\theta_R$, onset angle, sliding speed |
| Two-drop merge | mass/impulse conservation, bridge timing |
| Thin filament | survival and breakup radius/time |
| Curved solid contact | penetration, slip/pinning threshold |
| 1k/10k/50k/100k | GPU scan, neighbor, solver and render times |
| 720p / 900p / 1440p | p50/p95/p99 frame time and optical error |

and an explicit workload description. The test harness saves a simulation trace or deterministic input sequence and exports numeric errors and GPU timings as machine-readable data.

Measurements use GPU timestamp intervals per stage and CPU frame timestamps, with at least 60 seconds of sustained load after warm-up. Report medians, p95, p99, allocation footprint, and failure rates. FPS averages alone can hide stutter. For an M2 Air, repeat after reaching stable thermals; use an external Vulkan GPU as a distinct platform, not a replacement for M2 measurements.

## 9.2 Numerical error measures

A reference pressure error is evaluated as $|\widehat{\Delta p} - 2\sigma/R|/(2\sigma/R)$ for validated spherical droplets. Volume drift is $|V(t) - V(0) - V_{\text{in}} + V_{\text{out}}|/\max(V(0), \epsilon)$, with the error normalization and emission bookkeeping reported. Contact angles are estimated from the reconstructed interface against measured or published surfaces, not by reading the input material parameter back.

An isolated-drop rendering comparison must match the same physical radius, view, lighting and material coefficients as a reference or trusted offline renderer. Report silhouette IoU or chamfer distance, normal deviation where ground truth exists, and perceptual metrics only where images are registered. A lower LPIPS score does not prove physical accuracy.

## 9.3 Ablations

Minimum ablations are: (A) naive unsorted neighbors versus sorted neighbors, (B) PBF versus DFSPH/IISPH, (C) explicit tension versus coupled implicit tension/friction, (D) isotropic versus anisotropic reconstruction, (E) no thickness versus Beer–Lambert, and (F) global resolution versus view-adaptive rendering. For each, hold resolution, particle count, input trajectory and stopping tolerance fixed when possible.

## 9.4 Decision gates

**Gate 0:** exact physical formulas and units pass deterministic tests. **Gate 1:** GPU indexing preserves all IDs, works for sparse and clustered domains, and gives correct neighbor lists. **Gate 2:** a one-drop GPU solve passes Laplace and mass benchmarks with increasing resolution. **Gate 3:** contact hysteresis and coalescence pass their targeted tests. **Gate 4:** optical reconstruction resolves a tiny drop and a thin stream with stable silhouettes. **Gate 5:** the bounded M2 scene meets numerical gates while sustaining p99 below 16.67 ms. A negative gate result blocks claims of completion.

**FIGURE PLACEHOLDER**

`figures/performance.png`

Insert GPU-stage timing stacks versus particle count (1k, 10k, 50k, 100k) for an M2 Air. Include 16.67 ms frame-budget line.

**Figure 7: Frame-time breakdown (not measured).** Future real traces should include particle indexing, solve, contacts, surface reconstruction, shading, and graphics-system overhead.

**FIGURE PLACEHOLDER**

`figures/hardware-comparison.png`

Insert qualitative comparison at matched physics resolution: M2 Air vs discrete Vulkan GPU, with p99 frame time and energy/thermal notes.

**Figure 8: Cross-device quality/cost (not measured).** Hardware-specific speedups must not be inferred from unrelated published GPU measurements.

# 10 Algorithmic Implementation Contracts

## 10.1 The indexing contract

The first integration milestone is not a fluid surface but a verifiable spatial index. Its dispatch inputs are a bounded particle count $N$, an origin $\mathbf{o}$, cell width $c$, and positive cell dimensions $\mathbf{D}$. For particle $i$, the unsigned cell coordinate is

$$\mathbf{q}_i = \left\lfloor \frac{\mathbf{x}_i - \mathbf{o}}{c} \right\rfloor, \quad k_i = q_x + D_x(q_y + D_y q_z), \tag{11}$$

provided $0 \le q_d < D_d$ for each axis. Out-of-domain particles must carry an invalid sentinel and an explicit counter. Naively casting a negative coordinate to `uint` before checking bounds can alias distant particles into a valid cell and contaminate every subsequent neighborhood computation.

**Pass 0 (clear).** Reset the active cell histogram, scatter cursors, and live diagnostic counters. A dynamically allocated index domain must not leave stale counts from the previous frame. **Pass 1 (keys/count).** One invocation validates each particle and computes $k_i$, incrementing the histogram for valid cells. **Pass 2 (exclusive scan).** Convert cell counts into monotonic offsets with final offset equal to the number of in-domain particles. **Pass 3 (scatter).** Each valid particle obtains an atomic slot from a *separate* cursor buffer and writes its own ID into the sorted array. **Pass 4 (density check).** Each particle enumerates its 27-cell neighborhood, performing a strict full-coordinate check and comparing candidates against its kernel support.

These passes require synchronization with the proper shader-write and shader-read access masks [17]. The count, offsets, cursor, and sorted-ID buffers must have bounded capacities; an overflow condition is a detected error, never

**Table 2: Evidence ledger, current repository state.** Checks marked *not measured* must not be presented as experimental results.

| Component | Verified status (2026-11-04) |
|---|---|
| SI property reference functions | Unit tests passed on clean reset branch |
| GPU particle storage / scan | Not implemented / not measured |
| Coupled implicit drop physics | Not implemented / not measured |
| GPU boundary contact | Not implemented / not measured |
| Optical droplet reconstruction | Not implemented / not measured |
| M2 GPU workload / p99 FPS | Not measured |

**Table 3: Index invariants.** These are pass/fail requirements for GPU and CPU reference implementations.

| Invariant | Acceptance condition |
|---|---|
| Count | $\sum_c n_c = N_{\text{valid}}$ |
| Offsets | $s_0 = 0$, $s_{c+1} = s_c + n_c$ |
| Scatter | Every valid ID occurs exactly once |
| Cell identity | $k(\mathbf{q_i})$ agrees with CPU reference |
| Neighbor set | Exact pair set for $\|x_i - x_j\| < h$ |
| Overflow | Counter set; no out-of-range write |

undefined GPU memory access. The exclusive scan's terminal value is checked against $N$ minus invalid particles. If the sum differs, the solver must not consume the array.

An important cost comparison is $D_x D_y D_z$ versus $N$: in a mostly empty one-metre cube with millimetre cells, a dense histogram would be enormous relative to a few isolated droplets. The implementation must select a compact active domain, use a hashed/sparse cell index, or reject the request with a reported memory requirement. It must not allocate an unbounded dense grid from the world size alone. The first pass may support only explicitly bounded scenes while the sparse representation is built and tested.

## 10.2 Pressure projection: baseline equations

For a PBF reference, let the normalized density constraint be

$$C_i = \frac{\rho_i}{\rho_0} - 1, \quad \lambda_i = -\frac{\max(0, C_i)}{\sum_k \|\nabla_{x_k} C_i\|^2 + \varepsilon}. \tag{12}$$

This one-sided form avoids demanding a full bulk density at the free surface but must be paired with surface-specific forces rather than assuming all deficits are compressibility errors. The position correction is assembled from the symmetric gradients and applied in Jacobi iterations to avoid processing-order bias. A constant number of passes per frame is not acceptable evidence of convergence: the maximum density error, mean error, and the residual of velocity divergence must be tracked separately.

A DFSPH comparison then adds a projection based on velocity divergence and can reuse neighbor structure while its positions remain within the validated support. Rebuilding the grid at every sub-iteration without measuring displacement wastes bandwidth, but reusing a stale grid can omit newly adjacent neighbors. The decision criterion is whether the maximum corrected displacement exceeds a conservative safety margin, with diagnostics for missed potential pairs.

## 10.3 Unified solver feasibility

The coupled pressure/tension/friction formulation in [7] solves a substantially harder system than PBF. Treating its published *offline* examples as evidence of real-time feasibility would be misleading. We therefore separate correctness targets from practical execution: (1) implement and validate the energy/force components in a small deterministic reference, (2) port a GPU matrix-vector product or equivalent coupled iteration, (3) compare its residual and work per substep with DFSPH, and (4) choose a bounded-quality option for the M2 only if the measured error remains acceptable.

Preconditioning and termination matter more than the nominal solver name. Iterations must stop based on a convergence threshold or an explicitly flagged iteration limit. An aggressive fixed iteration count may create small droplets that appear frozen, jitter, or explode even when the main bulk remains incompressible. Any iterative method requires an error trace over at least two spatial refinements and multiple capillary numbers.

## 10.4 GPU memory ownership

We divide memory into (i) immutable material constants, (ii) live particle state, (iii) temporary sort/scan arrays, (iv) iterative solver scratch, (v) boundary acceleration, and (vi) optical image targets. Multiple in-flight frames must never overwrite a state buffer still consumed by a render pass. The simplest safe implementation serializes physics update and rendering within a frame using precise compute-to-graphics barriers, with a separate camera/scene uniform allocation per in-flight frame. Later overlap is optional, provided the extra buffered state and synchronization do not exceed the hardware budget.

The `VulkanFluid` API should surface three distinct outcomes: an accepted physical substep, a nonfatal capacity/backpressure event, and a solver failure that invalidates the state. A high FPS result is not acceptable if an overloaded run silently drops pending volume. Assertions and instrumented counters are retained in release builds where a GPU memory overrun would otherwise escape detection.

# 11 Optical and Perceptual Validation

## 11.1 Optical reconstruction is an inverse problem

The true visible interface is not directly stored by an SPH particle cloud. Screen-space methods infer a smooth front surface from projected particle coverage, often with thickness-dependent filtering [11]. The surface estimate must respect the 3-D geometry: two droplets at different depths should not be merged merely because their masks overlap on the display. Foreground geometry, water contact with a wall, and thin gaps require depth-aware neighborhood rejection.

The anisotropic reconstruction of Yu and Turk [9] suggests using local eigenvalues to classify a neighborhood. A nearly isotropic covariance often indicates a bulk-like sample; one small eigenvalue may indicate a thin sheet; two small eigenvalues suggest a filament. These are only statistical features and should be combined with the estimated particle support and boundary distance. A solitary particle has no reliable covariance, hence the analytic isolated-drop path.

## 11.2 Thickness and energy accounting

For a front-facing interface, refracted background light and reflected environment light should be weighted by a Fresnel term $F(\theta, \eta)$; the transmitted contribution is then attenuated by Eq. (10). A minimum useful compositor has separately stored front depth, integrated or backface thickness, normal, phase ID, and motion. Compositing in a linear-light space avoids exaggerated darkening or blown-out highlights from applying Beer–Lambert directly in display gamma.

The apparent refractive distortion of thin droplets is strongly influenced by their curvature. A fake uniform normal map can make a physically flat film appear raised. Conversely, excessive depth smoothing can make a tiny spherical droplet look like a featureless smudge. We therefore measure projected silhouette and estimate the normal field independently; an attractive specular highlight alone is not evidence of correct meniscus geometry.

A practical rendering-resolution fallback may produce the liquid at full-resolution depth and quarter/half-resolution diffuse thickness, with edge-aware reconstruction. This is only a candidate: whether it saves time without erasing subpixel droplets must be demonstrated. For microscopic dots, a conservative full-resolution analytic draw may be cheaper and more stable than processing them through a globally downsampled volumetric pass.

## 11.3 Color and display assumptions

The default material should use measured or explicit linear absorption and refractive index, not a color-tint parameter standing in for optics. A three-channel approximation is sufficient for the first real-time baseline, but it cannot model dispersion or wavelength-dependent scattering in detail. HDR scene color should remain linear until the host tonemap stage. Reflections and transmission must use correct surface orientation and background-depth rejection when the refracted ray would sample an occluding foreground object.

If no valid background sample exists, use a documented environment fallback instead of leaking unrelated screen pixels. Transparent geometry behind the fluid is not necessarily represented in an opaque scene-color buffer, so the compositor's treatment of other transparent objects must be stated. These limitations matter most for tiny droplets on bright high-contrast surfaces, where a one-pixel error is visually obvious.

### 12 Performance Falsification Strategy

The first useful benchmark is deliberately uninteresting visually: one thousand identical particles, held in a bounded domain, with the render pass disabled. This isolates the cost of GPU emission, cell indexing, scan, neighbor traversal and a single solver iteration. The next step increases particles while holding average neighbors constant. Only then should clustering, multi-scale dispersion, small droplets contacting surfaces, and the full optical pipeline be enabled.

**For a minimal GPU, reducing work is more important than maximizing arithmetic occupancy.** Skip inactive cells, reuse allocations, schedule only accepted physical substeps, and distinguish physics work from screen-space shading work. Do not assume that changing an SPH kernel from eight to sixteen iterations has a fixed cost; neighbor-list regularity and memory traffic can dominate. Likewise, do not apply desktop-specific subgroup tricks before querying the actual device. The resource and synchronization constraints are grounded in the Vulkan specification [17] and target-GPU memory principles [18].

**Table 4: Performance interpretation rules.** Each pattern suggests a distinct bottleneck, rather than an automatic reduction in physical detail.

| Observed regression | Investigate first |
|---|---|
| Index time scales with empty volume | Dense-grid histogram waste |
| Neighbor time grows superlinearly | Cell-size mismatch / clustering |
| Pressure time high at low $N$ | Dispatch count / solver iterations |
| GPU idle while CPU busy | Allocation, readback, submission |
| Fluid time grows with screen area | Optical pass, not SPH |
| P99 spikes, median stable | Resource lifetime / barriers / thermal |

We define a falsification rule for the proposed architecture: if a correct, bounded small-droplet GPU solve already exceeds the whole frame interval at the minimum target sample resolution, moving it to asynchronous compute or hiding it behind graphics does not make the simulation fundamentally real-time. The research program must then report a narrower supported scene or evaluate a different numerical model. The result should be documented as a limitation, not concealed by lower solver accuracy.

### 13 Implementation Plan and Scope

The existing `VulkanFluid` public namespace and `c build` static-library pattern are retained. The previous experimental blood and CPU-film implementation is archived separately and is *not* part of the proposed baseline. A small, headless material-property module, with formulas for capillary length, pressure and dimensionless numbers, is the first verified code artifact. Nothing in this paper implies that the proposed GPU kernels, render images or benchmark values have already been obtained.

**Phase I: GPU data path.** Implement persistent storage, emission queue, compaction, cell keys, histogram, exclusive scan, scatter, GPU correctness validation and timestamps. Prove no particle loss when capacity is reached.

**Phase II: solver comparison.** Implement PBF baseline and DFSPH/IISPH comparison with identical neighborhood data and time budgets. Quantify pressure residual, volume drift and divergent velocity. Only then implement a coupled capillary/contact solver adapted from the relevant literature.

**Phase III: microdrop physics.** Validate spherical-drop pressure, adhesion/pinning, two-drop coalescence and filaments. Ensure the GPU solver passes at more than one spatial resolution. Add conservation assertions for emission and detachment.

**Phase IV: renderer.** Add analytic isolated-drop shading, anisotropic connected-liquid surfaces, depth/thickness passes, Beer–Lambert absorption, Fresnel reflection and screen-space refraction. Check visual stability without relying on TAA or manufactured noise.

**Phase V: bounded real-time gate.** Measure M2 GPU passes, optimize sort and contact divergence, stage memory barriers accurately, and retain only changes that preserve

numerical quality. Release to `vulkan-game` only after the integration runs cleanly on actual hardware.

## 14 Limitations and Threats to Validity

The proposed solver is not a full molecular-scale simulation, nor can it represent arbitrary film rupture below its particle resolution. Contact-line hysteresis is strongly material- and contamination-dependent; validated constants must be measured or sourced. A mesoscopic SPH surface tension model can produce plausible images while failing the true Laplace pressure jump. A smooth reconstructed meniscus may hide an inaccurate solver. Screenspace refraction cannot reconstruct missing background radiance. Sparse neighbor grids can suffer from cell collisions, load imbalance and pathological clustering; benchmark suites must deliberately include these conditions.

Power, memory bandwidth and thermal throttling create different bottlenecks on fanless Apple Silicon and discrete GPUs. Published frame times from an RTX-class GPU or offline SPH paper cannot be relabeled as M2 expectations. Our proposed performance gates are aspirational until recorded on the named device. The detailed design deliberately exposes such gaps to prevent another visually attractive but numerically unreliable implementation.

## 15 Conclusion

A practical small-scale fluid simulator must be founded on capillarity, incompressibility and boundary hysteresis, then engineered for high-throughput GPU neighborhoods and physically credible rendering. We propose a GPU-resident pipeline in which a measured inexpensive pressure baseline is compared against strongly coupled pressure–surface-tension–friction models, while optical reconstruction treats isolated microdroplets differently from connected streams. The central engineering principle is to preserve physical state across quality/performance decisions: render LOD can change without liquid mass changing. The central scientific principle is equally strict: only reproducible pressure, contact, mass, image and frame-time measurements justify an accuracy or real-time claim. The open challenge is to achieve the desired error bounds within the M2's sustained rendering budget.

## References


[1] Markus Ihmsen, Jens Cornelis, Barbara Solenthaler, Christopher Horvath, and Matthias Teschner. Implicit incompressible sph. *IEEE Transactions on Visualization and Computer Graphics*, 20(3):426–435, 2014. doi: 10.1109/TVCG.2013.105.

[2] Jan Bender and Dan Koschier. Divergence-free smoothed particle hydrodynamics. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, pages 147–155, 2015. doi: 10.1145/2786784.2786796.

[3] Miles Macklin and Matthias M"uller. Position based fluids. *ACM Transactions on Graphics*, 32(4):104:1–104:12, 2013. doi: 10.1145/2461912.2461984.

[4] Xiao Nie, Leiting Chen, and Tao Xiang. Real-time incompressible fluid simulation on the gpu. *International Journal of Computer Games Technology*, 2015. doi: 10.1155/2015/417417.

[5] Alex Chow, Benedict D. Rogers, Steven Lind, and Peter Stansby. Incompressible sph (isph) with fast poisson solver on a gpu. *Computer Physics Communications*, 226:81–103, 2018. doi: 10.1016/j.cpc.2018.01.005.

[6] Nadir Akinci, Gizem Akinci, and Matthias Teschner. Versatile surface tension and adhesion for sph fluids. *ACM Transactions on Graphics*, 32(6):182:1–182:8, 2013. doi: 10.1145/2508363.2508395.

[7] Timo Probst and Matthias Teschner. Unified pressure, surface tension and friction for sph fluids. *ACM Transactions on Graphics*, 44(1):7:1–7:28, 2024. doi: 10.1145/3708034. URL `https://doi.org/10.1145/3708034`.

[8] Timo Probst and Matthias Teschner. Monolithic friction and contact handling for rigid bodies and fluids using sph. *Computer Graphics Forum*, 42(1):155–179, 2023. doi: 10.1111/cgf.14727.

[9] Jihun Yu and Greg Turk. Reconstructing surfaces of particle-based fluids using anisotropic kernels. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, pages 217–225, 2010. URL `https://diglib.eg.org/items/2d966af2-5428-41ca-b90c-2cf76c1f4b53`.

[10] Ryoichi Ando, Nils Th"urey, and Reiji Tsuruno. Preserving fluid sheets with adaptively sampled anisotropic particles. *IEEE Transactions on Visualization and Computer Graphics*, 18(8):1202–1214, 2012. doi: 10.1109/TVCG.2012.87.

[11] Wladimir J. van der Laan, Simon Green, and Miguel Sainz. Screen space fluid rendering with curvature flow. In *Proceedings of the 2009 Symposium on Interactive 3D Graphics and Games*, pages 91–98, 2009. doi: 10.1145/1507149.1507164.

[12] Orestis Vantzos, Omri Azencot, Max Wardetzky, Martin Rumpf, and Mirela Ben-Chen. Functional thin films on surfaces. *IEEE Transactions on Visualization and Computer Graphics*, 23(3):1179–1192, 2017. doi: 10.1109/TVCG.2016.2605083.

[13] Chenfanfu Jiang, Craig Schroeder, Andrew Selle, Joseph Teran, and Alexey Stomakhin. The affine particle-in-cell method. *ACM Transactions on Graphics*, 34(4):51:1–51:10, 2015. doi: 10.1145/2766996.

[14] Muhammad Waseem and Min Hong. Moving towards large-scale particle based fluid simulation in unity 3d. *Applied Sciences*, 15(17):9706, 2025. doi: 10.3390/app15179706.

[15] Duane Merrill and Michael Garland. Single-pass parallel prefix scan with decoupled look-back. Technical Report NVR-2016-002, NVIDIA Research, 2016. URL `https://research.nvidia.com/publication/2016-03_single-pass-parallel-prefix-scan-decoupled-look-bac`

[16] C. G. L. Furmidge. Studies at phase interfaces. i. the sliding of liquid drops on solid surfaces and a theory for spray retention. *Journal of Colloid Science*, 17(4): 309–324, 1962. doi: 10.1016/0095-8522(62)90011-9.

[17] Khronos Group. Vulkan guide: Synchronization examples. https://docs.vulkan.org/guide/latest/synchronization_examples.html, 2026. Accessed 2026-11-04.

[18] Apple. Tailor your apps for apple gpus and tile-based deferred rendering. https://developer.apple.com/documentation/metal/tailor-your-apps-for-apple-gpus-and-tile-based-defe 2026. Accessed 2026-11-04.

[19] Apple. Metal performance best practices guide. https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/, 2026. Accessed 2026-11-04.